

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

September 11, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	September 11, 2026	40 pages

Tiivistelmä — Referat — Abstract

Context. Radio access network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

Objective. The thesis compares modern C++ and Rust on a user equipment (UE) Context Management service modelled on a real Nokia 5G base-station control-plane component (gNB-CU-CP). Average-throughput comparisons leave four dimensions open: tail latency, the C++ defect classes Rust prevents at compile time, the overhead of its safety model, and the cost of incremental adoption.

Methods. One language-agnostic specification is frozen first, then implemented twice with matched algorithms, concurrency model, allocator, and build settings, and with forced-idiom gaps logged. A synthetic workload is increased to overload, both versions pass the same parity tests, and the evaluation adds fault injection and a foreign function interface (FFI) prototype.

Results. (1) Safe Rust rejects all seven injected memory-safety and data-race defect classes at compile time. In C++ the same seven build cleanly and show up only at runtime, if a sanitiser catches them. (2) Seven further classes fail in both languages. On two of them C++ tooling reports what Rust does not. (3) C++ can call the Rust service through a thin C interface. All `unsafe` stays there, and any extra time on a real call is at most 10.91% of one operation. (4) A full request spends only 0.250% on language work. Passing work between threads takes the rest, so that clock cannot attribute a gap to C++ or Rust. Timed on its own, Rust's operation is 1.6× faster. (5) Below saturation the paired test on the end-to-end median resolves 1.3 μs in Rust's favour. That gap is too small to bound a language effect. The Rust versions keep up with the offered load to 448 kops/s, the C++ versions only to 392. Once neither can keep up, C++ still finishes 1.7× as much work as Rust in 20/20 interleaved repeats with both allocators.

Conclusions. The main cost of adopting Rust is engineering effort and the learning curve, not run-time performance at nominal load. The gain is freedom from data races, not from concurrency defects in general. A nominal-load test would not expose the gap.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures iii

List of Tables iv

List of acronyms v

1 Introduction 1

1.1 Motivation . . . 2
1.2 Industrial context . . . 3
1.3 Problem statement . . . 4
1.4 Research questions and hypotheses . . . 4

2 Background: Systems Languages for Latency-Critical Software 6

2.1 RAN control-plane systems and the gNB-CU-CP . . . 6
2.2 Predictability and the two languages . . . 6
2.3 Research gap . . . 7

3 Research Methodology 10

3.1 The UE Context Management subsystem . . . 10
3.2 Research approach . . . 11
3.3 Methods per research question . . . 12
3.4 Fairness as a controlled variable . . . 13
3.5 Statistical treatment . . . 13
3.6 Reproducibility and research artifacts . . . 14

4 Design and Implementation 15

5 Experimental Setup and Evaluation 17

5.1 Testbed setup . . . 17
5.2 Workload and measurements . . . 18
5.3 Results . . . 18
5.3.1 Below saturation (RQ1, RQ3) . . . 19

5.3.2 Past saturation (RQ1) . . . 19
5.3.3 Resource use (RQ3) . . . 20
5.3.4 Safety via fault injection (RQ2) . . . 21
5.3.5 Migration (RQ4) . . . 22

6 Discussion **24**
6.1 Summary of findings . . . 24
6.2 Discussion on the research questions . . . 24
6.3 Revisiting the hypotheses . . . 26
6.4 Engineering guidelines for incremental adoption . . . 27
6.5 Where this study sits . . . 27
6.6 Ethical and professional reflection . . . 28

7 Conclusion and Future Work **29**

Statement on the Use of AI Tools **31**

Bibliography **33**

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 7
2.2 What the neighbouring studies control, and what they leave varying 9

3.1 UE-context lifecycle state machine 10

4.1 Shared-nothing, thread-per-shard architecture 15

5.1 The two workloads, and the lifecycle the mixed one walks 18
5.2 The load sweep: tail latency and achieved throughput 19
5.3 The end-to-end operation decomposed, against what the sweep can resolve 20
5.4 Divergence past saturation 20
5.5 Resource use, and the two effects inside peak RSS 21
5.6 Where the guarantee stops: the outcome matrix 22
5.7 The C++-to-Rust FFI migration seam 23

6.1 Where Rust's compile-time guarantee stops 25

List of Tables

3.1 How each research question is answered 12

List of acronyms

Each acronym below is also written out where the text first uses it.

Acronym	Expansion
3GPP	3rd Generation Partnership Project, the body that specifies the 5G standards
ABI	Application binary interface
AI	Artificial intelligence
AI-RAN	An architecture in which AI and radio access network workloads share one accelerated infrastructure
API	Application programming interface
CPU	Central processing unit
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
EU	European Union
FFI	Foreign function interface
FIFO	First in, first out
FNV-1a	Fowler–Noll–Vo hash function, variant 1a
gNB	Next-generation NodeB, the 5G base station
gNB-CU	gNB central unit
gNB-CU-CP	gNB central unit control plane: the system class studied here
gNB-CU-UP	gNB central unit user plane
gNB-DU	gNB distributed unit
HDR	High dynamic range, of the latency histogram
IMT-2030	International Mobile Telecommunications-2030, the ITU framework for 6G
ITU	International Telecommunication Union
MPSC	Multi-producer, single-consumer
NG-RAN	Next-generation radio access network
PHY	Physical layer
RAII	Resource Acquisition Is Initialization
RAN	Radio access network
RQ	Research question
RRC	Radio Resource Control
RSS	Resident set size
UE	User equipment, an end device such as a phone or a modem
UECM	UE Context Management, the subsystem reimplemented and measured here

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers. Functions that once ran on fixed-purpose, vertically integrated equipment are now re-implemented as programs on general-purpose machines. The radio access network (RAN), long the most hardware-centric part of the stack, is part of that shift.

In 5G, the base station, formally the next-generation NodeB (gNB), is split into a central unit and one or more distributed units. The central unit is split again into control-plane and user-plane functions [1, 2]. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces [3]. The control plane that results, the gNB central unit control plane (gNB-CU-CP), is a large, stateful, highly concurrent program. It tracks every connected device, each of which the standards call a user equipment (UE), runs Radio Resource Control (RRC) signalling [4], and coordinates mobility and handovers, all within strict latency limits.

Load at a busy site is high. A single site can run thousands of connection-management and mobility procedures per second, each of which has to complete within the millisecond-scale budget the 5G system sets for control-plane signalling [5]. A deployed control plane may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network. An outage or a tail-latency spike here is not a local defect. Every device the site serves is affected [6, 7].

This control-plane software is, today, written mostly in C++. The choice is a practical one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [10, 11], with performance intended to be competitive with C++ [12]. Whether that promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it de-

velops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service modelled on a real Nokia gNB-CU-CP component, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload.

Three terms recur throughout. The *gNB-CU-CP* is the control-plane software this work is about. *UE Context Management (UECM)* is the specific subsystem reimplemented and measured. *Tail latency*, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [6, 7].

1.1 Motivation

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN. Control-plane functions now run on general-purpose servers, so the software is longer-lived, more concurrent, and must meet its deadlines without the determinism of purpose-built hardware. Open-RAN disaggregation turns the base station into virtualised, multi-vendor software [3]. A latency regression or a memory-corruption defect is then a network-wide risk, and the language choice is an engineering decision.

The second trend is artificial intelligence (AI), which the network once used to tune itself and now carries as a workload in its own right [13]. International Mobile Telecommunications-2030 (IMT-2030), the 6G framework of the International Telecommunication Union (ITU), lists that second role as one of its usage scenarios [14]. The architecture forming around it is AI-RAN, in which AI and radio work share one accelerated server [15]. A control plane there no longer has the host to itself. It shares the central processing unit (CPU), memory, and cache with inference work it does not schedule. When that neighbour takes capacity, the control plane saturates sooner: it stops keeping up with demand at a lower load than it would alone. Chapter 5 finds the widest gap between the two languages past that point.

The third is the shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them [8, 9, 16, 17]. Independent work argues that memory safety now needs standardised requirements if it is to be procured at all [18]. Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 [19, 20, 21]. The Act names no programming language. Direct evidence for a RAN control plane is still missing. Providing it is the motivation for this work.

1.2 Industrial context

This thesis is carried out at Nokia. The company is one of the small number of vendors that build, maintain, and evolve production RAN software end to end. The five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [22, 23]. The subsystem it studies, UECM, is modelled on a real gNB-CU-CP component rather than a textbook example.

In the architecture of the 3rd Generation Partnership Project (3GPP), the gNB-CU-CP is the control-plane half of the central unit. It hosts RRC and manages the UE context lifecycle [1, 4], under the millisecond-scale budgets the 5G system sets for control signalling [5]. Chapter 2 places it in the wider architecture. Because it is written in C++, it inherits that language's performance and its memory-safety and concurrency hazards. The three trends of Section 1.1 meet here. Software of this class runs on live operator networks. The language choice is a production concern.

UECM is small enough to reimplement faithfully and reason about in full. It still carries the properties that make control-plane code hard. The implementation follows a public, frozen specification, not proprietary code, so readers outside Nokia can reproduce the comparison.

A decision with a generational horizon. A language choice in software of this class outlives the generation it was made for. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work. The ITU asks for technology proposals in early 2029 and submitted specifications by mid-2030 [24]. That work covers migration from 5G and interworking with it. A control plane written now is maintained alongside the next generation, not retired. IMT-2030 still names security and resilience as a design principle and still carries the ultra-reliable low-latency scenario in strengthened form [14].

The measurements here use a current-generation subsystem and published requirements. No claim is made about requirements that have not been written.

A concrete failure mode. Consider a mobility storm in which many UEs hand over at once. A UE context freed on release while an in-flight signalling message still references it is a use-after-free. In C++ that compiles and may pass ordinary tests. It can then corrupt another UE's state or crash the worker under high concurrency. The same defect can show up as a tail-latency spike instead of a crash. In safe Rust the design does not compile:

ownership and borrowing reject the dangling reference. That is why this thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload), not average throughput alone.

1.3 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation, which are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [6], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput), whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [12, 25], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.4 Research questions and hypotheses

The thesis is organised around four research questions, ordered to put predictability and migration ahead of raw average performance:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99, p99.9 and p99.99 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, foreign function interface (FFI) interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

Three hypotheses are written down before any measurement.

H1 For this workload the two languages are performance-equivalent below saturation, with any difference only in the far tail.

H2 Safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ only tries to catch after the program is built.

H3 The extra time when C++ calls into Rust is small next to the cost of the operations themselves, so the real cost of adding Rust a piece at a time is engineering work, not run-time speed.

Chapter 6 revisits all three against the evidence.

The defect list for RQ2 includes classes safe Rust is expected to prevent and classes it is expected not to. The study can then find where the safety guarantee ends, instead of only showing that it holds.

A previous study already estimated that boundary from a catalogue, not from compiled programs. It took the Common Weakness Enumeration (CWE), a long list of bug types, and marked each type as one that safe Rust would prevent or would not. It then counted real vulnerabilities in 68 open-source C projects. 58.2% of those classified vulnerabilities sat in types that safe Rust would have made almost impossible [26]. That share is a best case: it assumes the new code stays in safe Rust. A type counts as prevented if the mapping says so, even if no compiler has rejected a program. RQ2 asks the same question of programs that are compiled and run. That is how it can say when the remaining types are caught, and what it costs to catch them.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The next-generation radio access network (NG-RAN) architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE [1, 2, 4]. Such control-plane systems must satisfy demanding non-functional requirements: very high event rates (UE mobility, signalling storms), strict and predictable latency bounds that the 5G service and RAN specifications quantify as key performance indicators (KPIs) [5, 27], long-lived software with continuously evolving complexity, and heavily concurrent workloads. In these systems the worst-case and tail behaviour, rather than the average, determines whether deadlines are met, which motivates the predictability focus of this thesis.

Functionally, the gNB is disaggregated along two independent axes. The first is a *central/distributed* split: a central unit (gNB-CU) hosts the higher layers of the protocol stack, while one or more distributed units (gNB-DU) host the lower, more time-critical layers close to the radio. The second is a *control/user-plane* split inside the central unit, separating the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP [1, 2, 28]. Open-RAN initiatives standardise these interfaces further so that each part can be supplied by a different vendor and run as software on commodity hardware [3]. Figure 2.1 places the subsystem studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP: it owns the per-UE control-plane state and mediates the RRC signalling and mobility procedures, which is why its correctness and timing behaviour are representative of the control plane as a whole.

2.2 Predictability and the two languages

This section states what the study takes as given. Predictability, not average throughput, is the governing property of a latency-critical service, so the quantity to measure is the

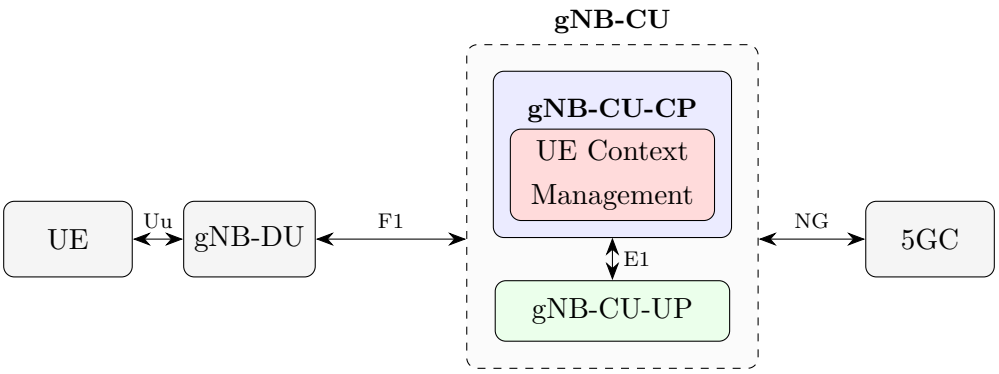


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. UE Context Management is highlighted inside the gNB-CU-CP.

tail of the distribution and not its mean, and measuring it correctly constrains the harness as much as the metric: an open-loop schedule is required, because a closed-loop one stops issuing requests while a stall is in progress and so never records the latency of the stall.

C++ supplies the control the domain demands and leaves memory and concurrency safety to the programmer, with sanitisers and review standing in for a guarantee. The standards committee's work on enforced safety profiles is neither standardised nor generally implemented. Rust's ownership and borrowing discipline rules out lifetime and aliasing errors at compile time, and its `Send` and `Sync` traits rule out data races. That is a narrower claim than the phrase "thread safety" suggests. It says nothing about lock-acquisition order, about the atomicity of a composed operation, or about leaks, and it is suspended wherever a program writes `unsafe`. Both languages can express the same concurrency structures, and a comparison that lets the structure vary with the language measures the structure. The two meet through a C application binary interface, which is the mechanism any incremental migration has to use and the reason a seam concentrates whatever `unsafe` a migration needs.

2.3 Research gap

The preceding sections set up the gap. The RAN control plane is a soft-to-firm real-time, highly concurrent domain where tail behaviour under overload, not average throughput, governs the quality the network delivers. C++ supplies the control and performance the domain demands, but leaves whole classes of memory and concurrency defects to best-effort detection after the fact. Rust promises to prevent those classes at compile time at no run-time cost, and offers a credible incremental adoption path through the C application binary interface (ABI).

The domain is not unstudied, and the volume of prior work is easy to underestimate because almost none of it is indexed. At least five vendor-hosted master's theses, all carried out at Ericsson, have compared candidate languages for RAN and telecom software. One weighs runtime cost against development efficiency across C++, Rust, and Go [29]. One asks whether Rust could replace C in RAN node management [30]. One builds a thread-per-core packet processor on asynchronous Rust and measures it against the Data Plane Development Kit (DPDK) application it mirrors [31]. One measures median and 99th-percentile latency for control-plane procedures across five asynchronous programming frameworks [32]. The last builds matched C++ and Rust implementations of an Abstract Syntax Notation One (ASN.1) decompiler and compares them statistically [33]. All five are directly relevant, and two report differences unfavourable to Rust.

Three of them vary the language together with the runtime and the concurrency model, so a measured difference cannot be attributed to the language rather than to the framework built on it [29, 31, 32]. The effect sizes reach a factor of five, which is an indication of how much of the measurement the runtime owns. The remaining two hold the design closer to fixed but report execution time and peak memory rather than tail behaviour under rising load [30, 33], and the closest of them, Emberg and Wadsten, measures a stateless transformation rather than a stateful, concurrent subsystem. Figure 2.2 places the five vendor theses beside the general-purpose comparisons [34] and the matched systems-software pairs [35, 36]. The pattern it shows is that each group holds some of the properties a controlled answer would need, and none holds all.

The concentration of that evidence is itself part of the gap. All five studies come from one vendor, and their conclusions have never been tested against another organisation's software, targets or conventions, so what is known about this question is what one company's frameworks report about it. The published record on the other side is emptier: there is no study of Rust among the indexed publications of the vendor hosting this work, and the nearest peer-reviewed position from that side argues for making memory-error sanitisation practical rather than for replacing it with a compile-time guarantee [37], which is the strategy this thesis tests against. The paragraphs below compare the five with this study directly.

What is therefore missing from the literature is *direct, controlled* evidence for a representative control-plane subsystem: Whether Rust actually holds its tail latency and predictability under overload against modern C++ *when the concurrency model, algorithms, and data structures are held fixed*, exactly which defect classes it eliminates and at what stage the remainder are caught, what its safety model costs in throughput and resources, and what incremental adoption costs in practice. Closing this gap is the purpose of the

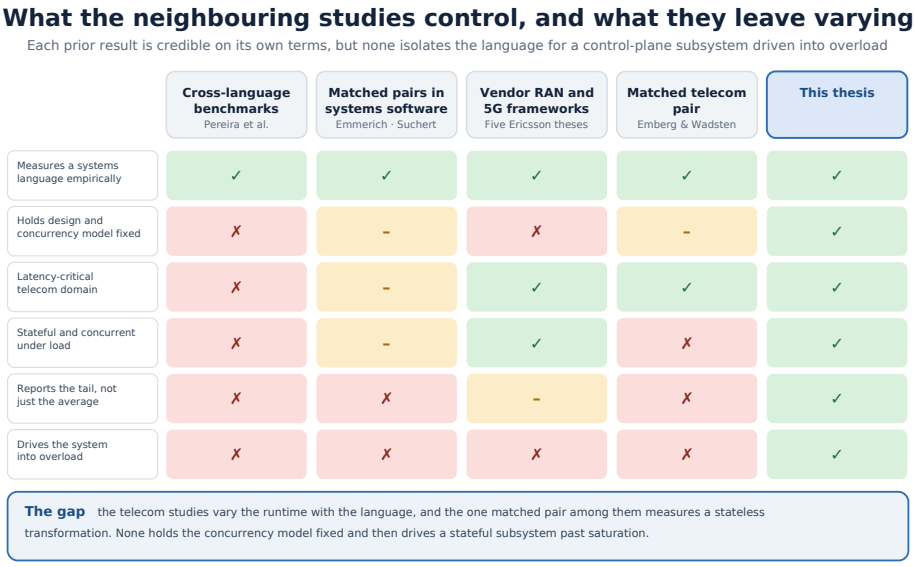


Figure 2.2: What the neighbouring studies control, and what they leave varying. Each row is a property a controlled answer would need, and each column is a group of studies. The vendor-RAN column is the five Ericsson theses, including both Olsson studies.

four research questions. The next chapter grounds them in the specific subsystem, defect classes, and timing properties at stake, and sets out the methodology by which they are pursued.

3 Research Methodology

The preceding chapter set out the gap this study closes. This one sets out how. It begins with the subsystem the work is carried out on. It then states the research strategy, the method for each question, the controls that keep the comparison fair, the statistical treatment, and how a third party can rebuild the results.

3.1 The UE Context Management subsystem

The subsystem under study is a UE Context Management service. It is modelled on a real Nokia gNB-CU-CP service domain that owns the UE context lifecycle, RRC procedure execution [4], and mobility and handover. Those are the functions the 3GPP architecture assigns to the control-plane part of a disaggregated gNB (Section 2.1). Figure 3.1 shows the UE-context lifecycle both versions encode the same way. The mixed workload of Section 5.2 walks each UE through that machine.

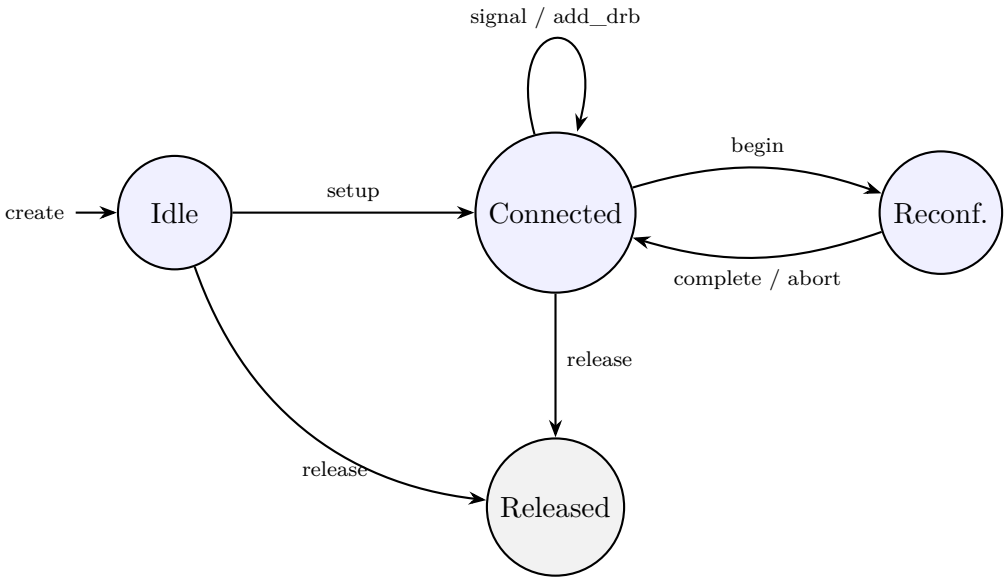


Figure 3.1: UE-context lifecycle state machine. Operations invalid for the current state return an error instead of changing state.

UECM is core RAN layer-3 work. It carries the properties that make live control-plane code hard: per-UE mutable state, a strict operation lifecycle, high concurrency across UEs, and the latency targets the 5G system sets for control-plane procedures [5, 27]. Conclusions about how a language handles those properties transfer to other stateful,

concurrent control-plane services. The subsystem is also small enough to specify in full and implement twice. That is what makes a parity-tested comparison possible.

The interface is small and built around the lifecycle. It can create and release a context, set up a connection, handle an in-session signalling message, add a data radio bearer, begin, complete, or abort a reconfiguration, and look up a context's state under load. Each operation is defined for each state of Figure 3.1. An illegal operation returns a typed error and leaves the context unchanged. That closed set lets the same tests and the same workload drive both versions along the same paths.

RQ2 needs a catalogue, not a loose list of hazards. One is built from this operation set: 14 classes in two groups of seven. Seven come from inside the region safe Rust claims to govern. Seven were chosen because safe Rust is expected not to prevent them. Each has its expectation recorded before the programs were run. Section 5.3.4 reports what each one did.

The tail is the metric because of two properties of this subsystem. One user-visible action, a call setup or a handover, becomes many dependent control-plane operations. The slowest member gates completion. A busy site drives thousands of such procedures per second, so a percentile far out in the distribution is reached often enough to matter. The failure shape is the second reason. A service of this design tracks its offered load with flat latency until a worker saturates. Then latency jumps by a huge step in one increment of load. That is why this study sweeps past saturation.

3.2 Research approach

The study runs experiments. It also builds the thing it measures: a matched pair of versions, plus the harness that drives them. Both are built from one frozen specification. Evidence comes from three separate methods. A load sweep answers RQ1 and RQ3. Controlled fault injection answers RQ2. An FFI migration prototype answers RQ4. A weakness in one of them, such as how much the host can move the tail, does not carry into the conclusions as a whole.

Each question asks how one outcome depends on one variable, the language. The language has to be the only planned gap between the two objects under test. That rules out the commonest form of language comparison, where two programs written on their own are measured and the gap is taken to be the language. Such a gap also carries the design, the libraries, and the skill of their authors. Building both sides from one specification removes that confound. Where a question can be answered with numbers, it uses numbers. Where

it cannot, the answer is still by a fixed plan, in words. That is how this study treats the kind of guarantee in RQ2 and the application programming interface (API) design effort in RQ4.

The closest prior work is the five vendor-hosted theses of Section 2.3. They appear in no citation database. They had to be found by searching the institutional repositories directly. The same searches found no Finnish thesis on Rust in a radio-access setting, and no indexed publication mentioning Rust from the vendor hosting this work. The search returned no study that measures the tail of a Rust and a C++ version of the same stateful, concurrent control-plane subsystem under overload.

Table 3.1 maps each research question to the method, artefact, data and metric that answer it. RQ1 and RQ3 share H1, worded to cover a speed tie in both the latency and the resource sense. That wording turns out to be the hypothesis's main weakness. RQ2 does not depend on the host. RQ4 depends on it along one axis: build time is a timing, and so it is read as a ratio between two toolchains measured in turn on one machine.

Table 3.1: How each research question is answered. RQ1 and RQ3 carry two methods rather than one: the sweep measures the round-trip a deployment would experience, and the split of the core from the handoff measures the part of it the language is responsible for.

RQ	Method	Artefact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9/p99.99, and maximum vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak resident set size (RSS), CPU, throughput
RQ1, RQ3	Core called directly, then through the service	Matched microbenchmark pair	operation vs. cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, three C++ sanitisers, `rustc`, Loom/Shuttle, Miri	compile-time / runtime / undetected
RQ4	FFI migration prototype	C++ host + Rust C-ABI seam	seam size, `unsafe` scope, per-call cost

3.3 Methods per research question, in outline

RQ1 and RQ3 are answered by an open-loop load sweep from light load to well past saturation. Open-loop means the generator keeps to its send schedule instead of waiting

for replies [38]. The four build variants are interleaved inside each repeat. A second timing splits the core operation from the cross-thread handoff around it. The end-to-end round-trip bundles two costs of two kinds, and only one of them can differ between the languages. A third timing takes the contended queue and completion primitives on their own, so the overload cause can be tested and not merely argued.

RQ2 is answered by controlled fault injection [39]. Each defect class is written as a small self-contained program in both languages. The C++ side is built and run under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer. The safe-Rust side is compiled and, where it compiles, run under a bounded timeout. No static analyser is run, on purpose. The C++ columns then report the tooling this study ran and not a ceiling on what C++ tooling could find. RQ4 is answered by a working prototype in the direction a real migration takes: a C++ host calling an unchanged Rust core across a C ABI. The per-call cost is measured on a real operation and not inferred from an empty crossing. Cross-language memory-management defects concentrate at that seam [40].

3.4 Fairness as a controlled variable

A language comparison is only as good as the variables it holds fixed. The specification is frozen before either version exists. The concurrency model, the allocator and its arena count, the optimisation level, the hash function, the histogram algorithm and the load schedule are matched across the two sides, never tuned per side. Where the two languages could not be written alike, the gap is logged instead of resolved. The two sides are shown to match by diffing conformance transcripts operation by operation. One row of that table has no control against it: one person wrote both versions and is not equally expert in both. Section 6.5 treats that as the study's main internal threat.

3.5 Statistical treatment

The statistical design is fixed in advance. The family of tested comparisons is listed ahead of the campaign, so a load picked after seeing a sweep cannot enter it. Each comparison is an exact paired permutation test over 20 interleaved repeats with a bootstrap confidence interval [41], Holm-corrected across the family [42]. The floors the test can reach (0.0000019 uncorrected, 0.000027 corrected) are quoted so that a null can be told apart from a math limit. Two quantities are kept out of the family. One is jitter from one call to the next, which no research question asks for. The other is the pooled p99.99 far tail.

Its unit is the operation, not the run. It then has no across-repeat interval to test.

3.6 Reproducibility and research artifacts

A third party can rebuild the results without access to Nokia's private code. The versions are driven entirely by the public frozen specification. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published with the thesis. Each plot of measured data, and each quoted number in the results, is rebuilt by one script (`bench/refresh_thesis.sh`) directly from the raw logs. That claim does not cover the hand-drawn schematics, which explain a concept rather than report a result. And running the same scripts over another host's campaign rebuilds the artefacts by the same procedure, not with the same values.

The logs that script reads are part of the package. Each campaign has its own script and its own macro prefix, so a figure from one cannot reach a sentence about another. Each run's host setup is recorded and stored beside the campaign data. Nothing in the results chapters is transcribed from those files by hand.

4 Design and Implementation

Nothing is built until the specification is frozen. It fixes the public API, the UE state machine, the data structures and their algorithms, the locking and shard grain, and the concurrency model. Both versions follow it. The test then times the language, not a gap of design or of skill. The specification is itself a deliverable. It is the study's main control against writer bias.

One concurrency model is used on both sides, shared-nothing and thread-per-shard. That structuring is known to change tail latency by itself [43]. The identifier space is partitioned into shards. Each shard is owned by a single worker thread that alone reads or mutates it, so no lock guards context state. Callers submit a request to the owning shard's bounded queue and wait on a single-use completion handle. The queue blocks when it is full. That is deliberate. A drop policy would hide overload, and this study needs overload to appear as latency. The only shared mutable state either version permits is one atomic counter. No control flow depends on its value. Figure 4.1 shows the resulting data flow, with one multi-producer, single-consumer (MPSC) queue per shard that its worker drains first in, first out (FIFO).

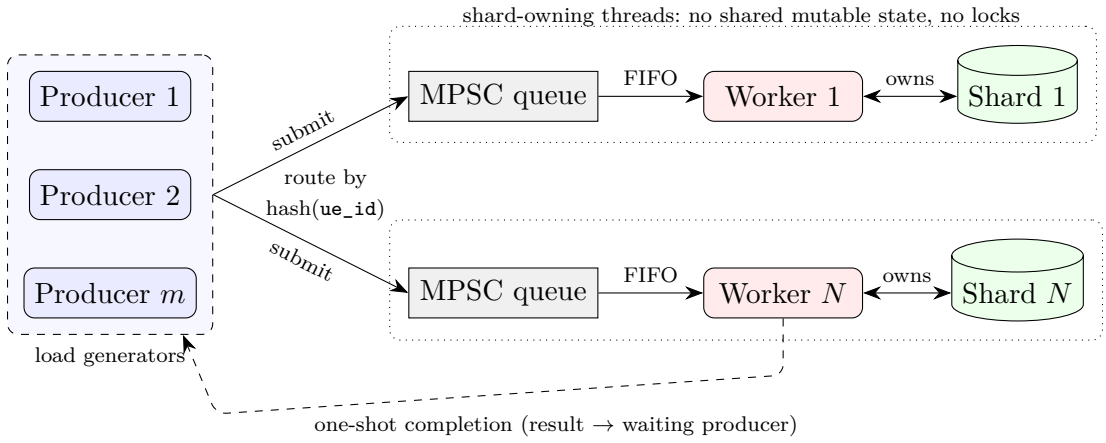


Figure 4.1: Shared-nothing, thread-per-shard architecture. Producers submit to a bounded MPSC queue per shard, and that shard's worker drains it.

Fairness is enforced below the design as well as in it. Both builds link the same `jemalloc` and keep a system-allocator build as a control. Both compile at matched optimisation. Both record latency through one high-dynamic-range (HDR) histogram algorithm set up the same way [44]. Hashing is pinned on both sides, SplitMix64 for shard assignment and FNV-1a, the Fowler–Noll–Vo hash, inside the map. C++'s default hash of an integer

is often just the number itself. Rust's is strong on purpose. Left as they came, the two standard libraries would have been the measurement.

Where the languages cannot satisfy a requirement the same way, the gap is written into a forced-idiom log and then measured. Two of those gaps carry the overload result of Section 5.3.2. One is the bounded queue, hand-written in C++ against Rust's `sync_channel`. The other is the completion handle, `std::promise` against a fresh `mpsc` channel per request. A third carries the core-operation result of Section 5.3.1, since the per-shard container is each language's own standard hash map. Rust takes its bounded blocking queue from `sync_channel`. C++ through C++23 has no such primitive, so it assembles one from a mutex and two condition variables. C++ has no payload-carrying enumeration in this idiom either. A request becomes a tagged struct holding each argument field any operation might need, 160 bytes against Rust's 96. The net result is a Rust core 18% longer in code lines whose total cyclomatic complexity is 15% lower [45]. The Rust library contains no `unsafe` at all.

Parity is tested and not assumed. Language-neutral vectors run in both versions and their transcripts are diffed operation by operation. A cross-diff cannot tell "both correct" from "both wrong in the same way". A third version in Python acts as an independent oracle. It has to reproduce all 223 hand-derived expectations before it may generate any of its own. A campaign of roughly 793 000 generated operations found no divergence. The C++ side runs under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer. The Rust side runs under Miri. A clean sanitiser result is bounded by the code that actually ran.

5 Experimental Setup and Evaluation

The chapter first names the machines and the workload, then reports the results. Setup questions come before any comparison. The findings are grouped by research question.

5.1 Testbed setup

Some results follow from the compilers and the source. Others depend on the machine. Each performance number here comes from one dedicated bare-metal server: a single-socket AMD EPYC 7443, simultaneous multithreading off, running Ubuntu 24.04.4 LTS on kernel 6.17.0-23-generic. The frequency governor is fixed at performance with boost disabled. Cores 2–17 carry the benchmark. There is no hypervisor. The kernel runs tickless on the pinned cores (`nohz_full`). The study does not use `isolcpus`. That flag also drops the cores from each scheduling domain, and the pinned set then collapses onto one core.

The toolchain is not the host's. Both versions are built and run inside a pinned container image carrying Rocky Linux 8.10 (Green Obsidian) with `g++ 15.2.1` and `rustc 1.97.1`. The image pins the `gcc-toolset` major version and not the release within it, so the earlier guest campaign compiled with `g++ 15.1.1`, one GCC 15 release earlier. The container also takes the kernel from its host. The two campaigns then differ in the hardware, in the kernel (6.17.0-23-generic against 7.0.2-1.el8.elrepo.x86_64), and in that compiler release.

Until late in the work the campaign ran on a guest instead: a Intel(R) Xeon(R) CPU E5-2680 v4 guest (kvm) with no core isolation. On that host the 95% confidence interval on a single variant's p99 spans 1480 μs, while the four variants' p99 values differ by 293 μs. The spread on one swamped the gap between all of them. That campaign is not discarded. It shares this one's design exactly, so the two can be set side by side to ask which findings belong to the code. A third sweep on a second bare-metal AMD EPYC 7443P uses a different shard count, load grid, repeat count and toolchain, so it tests whether the findings survive a change of design rather than a change of machine.

5.2 Workload and measurements

The workload is synthetic and spec-justified. A deterministic driver walks each UE through the connection lifecycle the 3GPP specifications define [1, 4]. The *signal* workload preloads connected UEs and hammers the hot path. The *mixed* workload walks each UE through context creation, setup, signalling, bearer setup, reconfiguration or handover, and release. A fixed draw order makes the per-operation-kind counts the same in both languages for a given seed. That doubles as a parity signal. Figure 5.1 shows the two workloads and the sequence the mixed one walks.

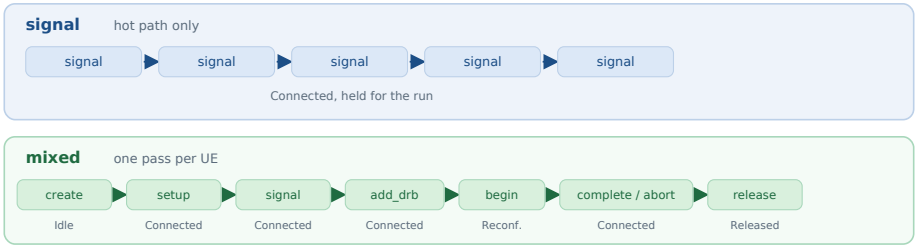


Figure 5.1: The two workloads, and the lifecycle the mixed one walks. Signal isolates the hot path, and mixed walks each UE through the procedure sequence drawn below.

The sequence of operations is justified. The proportions are not. How often a real cell hands over depends on deployment and time of day, and no public figure fixes it. A mix that does not match a real cell shifts both languages equally. It bounds how far the numbers travel, not the fairness of the comparison. The send schedule is open-loop and the histogram is corrected, so a stall introduces no coordinated-omission bias [38, 44]. Each sweep point is repeated 20 times. The four language and allocator variants are interleaved inside each repeat. Each point is a median with a bootstrap 95% confidence interval [41, 46].

The fixed load is fixed at 56 kops/s and not derived. The rule this study began with, the busiest swept load holding p99 under 5 ms, located the guest's noise floor. On a quiet host it climbs until it collapses onto the saturation knee. It is kept as a diagnostic only. The measured throughput plateau is 392 kops/s, the lowest of the four, reached at an offered 392 kops/s.

5.3 Results

The safety and most of the migration results do not depend on the host. The performance and resource results do. Each figure below is measured on the host of Section 5.1 unless

stated otherwise.

5.3.1 Below saturation (RQ1, RQ3)

Below saturation the four variants sit within 2 μs of one another at the median, 2.1%. The paired test resolves a difference of 1.3 μs in Rust's favour under both allocators. Figure 5.2 draws the sweep behind that. Higher up the distribution the ordering can now be read. The widest interval on a single variant's own p99 spans 1 μs against 2 μs of spread between all four, so the spread on one is smaller than the gap between them. On the guest the same two quantities were 1480 μs against 293 μs, the wrong way round.

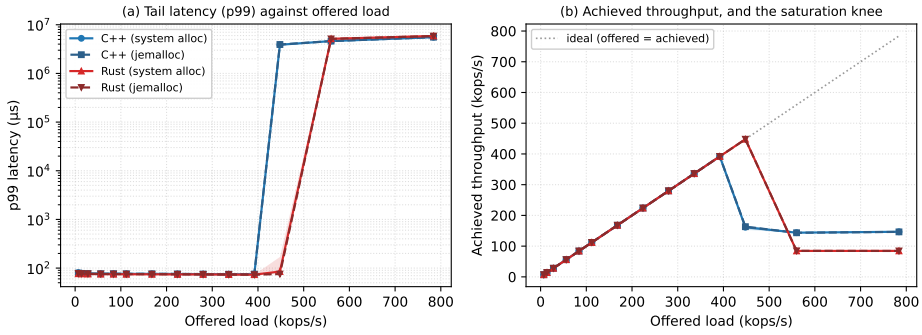


Figure 5.2: The load sweep, both panels against offered load. Lines are medians over 20 repeats, and the bands are bootstrap 95% confidence intervals.

The far tail resolves here too, from the repeats' histograms pooled into 3 360 051 to 3 360 084 operations per variant. Both Rust variants sit below both of the others by up to 87 μs. A second bare-metal host at another shard count agrees, running 147–228 μs for the C++ variants against 88–115 μs for the Rust ones, without overlap. A second limit is structural. No host would fix it. An end-to-end round-trip is 0.250% the work the language performs. The rest is a cross-thread handoff charged to the operating system. Once per-operation work is small, communication across isolation boundaries dominates [47]. This sweep cannot bound a language effect in either direction, which Figure 5.3 sets against what it can resolve. Measured on its own the core operation is 1.6× faster in Rust. The likely cause is the standard hash map's layout, not anything about safety.

5.3.2 Past saturation (RQ1)

Once the offered rate outruns the service, two differences appear. They stop meeting the offered rate at different loads: the Rust arms hold to 448 kops/s and the C++ arms to 392, so at 448 kops/s Rust still serves 448 kops/s where C++ has collapsed to 162. That load

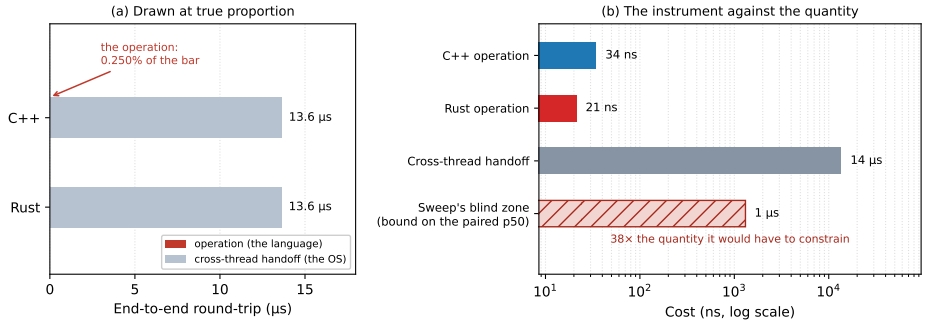


Figure 5.3: Where an end-to-end round-trip's cost goes. **(a)** The round-trip at true proportion. **(b)** Both operation measurements sit inside the sweep's blind zone by a factor of 38.

level is described and not tested, because the load was found after the data was in hand. Deeper, at 784 kops/s offered, the ordering reverses. C++ sustains 147 kops/s where Rust sustains 84, a factor of 1.7, in 20/20 repeats under both allocators with the Holm-corrected p at the floor of the whole test family. The slower variant is not simply doing less work. It draws 1.8 times the CPU per completed operation, which is the signature of effort spent without completing anything. Taking the bounded queue and the completion handle on their own does not reproduce that ordering at significance, so the cause sits in the wider handoff path. This is the largest language effect in the tested family. A comparison that stopped at a load the service still meets would have read the other way. Figure 5.4 sets what each variant still completes beside what it spends completing it.

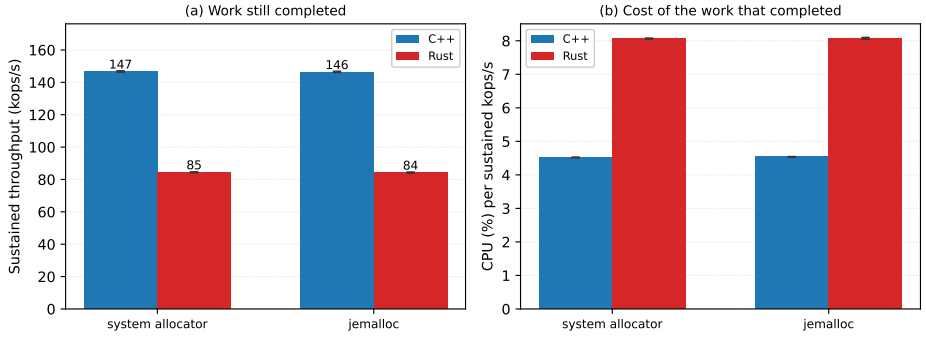


Figure 5.4: Behaviour at 784 kops/s offered. Bars are medians over 20 repeats with bootstrap 95% intervals, grouped so the two bars in a group differ only in language. CPU in (b) is summed across threads.

5.3.3 Resource use (RQ3)

Resource use separates the variants where latency could not. Two effects of comparable size sit in the resident set. The Rust build costs 1.4–2.5 MiB more peak resident set

than the C++ build under either allocator. Linking `jemalloc` at all costs 1.5–2.4 MiB in either language. Both are real, and Figure 5.5 puts the pair beside the peak RSS they decompose. The language effect keeps its direction and scale on all three machines measured. The allocator's was about ten times larger on the guest this study used first (32.5–34.3 MiB, about 13 times the language effect), so their ratio is a property of the machine. The language difference is a fixed cost, not a per-context one. The quiet host locates it. `jemalloc`'s own accounting there puts +2.07 MiB more in arena bookkeeping and −0.16 MiB *less* in live allocation, and its resident bytes are −1.92 MiB lower while the process peak is +1.74 MiB higher. The footprint then sits outside the allocator, in the runtime a Rust process carries before it allocates. CPU inverts with the regime. At the representative load of 56 kops/s Rust draws at most 5.1 percentage points less. Once the service no longer copes, CPU per completed operation is much higher.

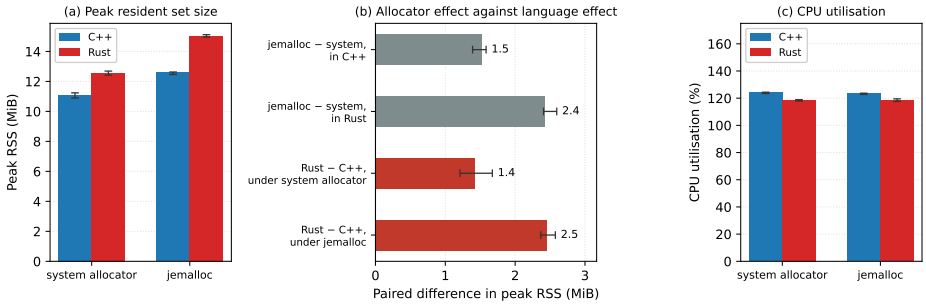


Figure 5.5: Resource use at 56 kops/s, grouped by allocator. Bars are medians over 20 repeats with bootstrap 95% intervals, and CPU in (c) is summed across threads.

5.3.4 Safety via fault injection (RQ2)

Safety is measured by injecting 14 defects. Seven come from inside the region Rust claims to govern. Seven were chosen because safe Rust is expected *not* to prevent them. Each one of the first seven is a compile error in Rust. It cannot be written at all. In C++ each is valid code. The defect appears only if a test drives it, and no single sanitiser covers them. The use-after-move is the case where no sanitiser reports anything: a moved-from object is valid but unspecified, so there is nothing to detect and the identity is simply lost. All seven boundary cases compile in safe Rust and misbehave. On two of them the C++ tooling is the one that reports. LeakSanitizer flags the reference cycle that Rust leaks in silence, because leaking is safe in Rust by explicit design [48, 49]. UndefinedBehaviorSanitizer flags the overflow that Rust defines to wrap. ThreadSanitizer catches the lock-order inversion in 5 of 12 runs, a rate that is itself unstable. It builds its lock-order graph from acquisitions that succeed. A run whose first inversion blocks never closes the cycle.

The seam double-free is the in-guarantee double-free rewritten through `unsafe`, not a new class. Its C++ half is unchanged, so only the Rust column moves. Two Rust model checkers do report both original boundary defects. Loom explores interleavings in a sound way. Shuttle samples the ones Loom cannot reach [50]. Each had to be brought in against the checker's own primitives first, and shrunk to model size. Miri reports the seam double release, but it interprets Rust and cannot execute the C++ host the defect ships behind. That half of the boundary is caught at test time, and for the concurrency cases only after a rewrite. Figure 5.6 draws the whole catalogue as an outcome matrix. Resource Acquisition Is Initialization (RAII) removes the ordinary missed-unlock. It does not remove the class: both forgotten-guard programs hang, and neither language's tooling said so.

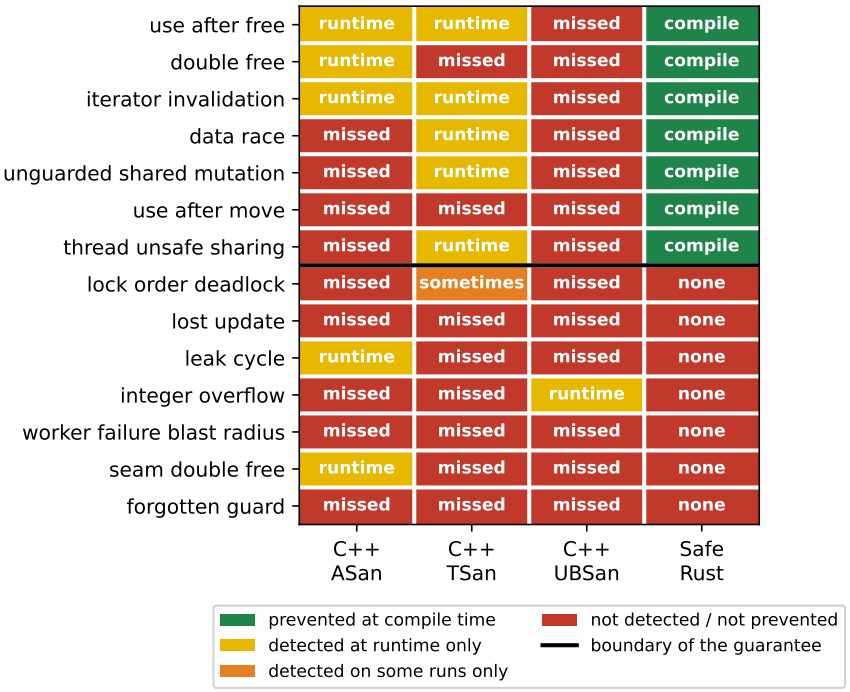


Figure 5.6: The fault-injection outcomes as a matrix, generated from the taxonomy log.

5.3.5 Migration (RQ4)

The migration seam is small on each axis measured: 11 C entry points in 231 lines of Rust. All raw-pointer work is confined to 9 functions. The core is left free of `unsafe`. No cost is detected against the same operation called directly. That bounds any per-operation tax at 10.91% of one operation, and Figure 5.7 shows the structure that bound belongs to. Three adoption costs beyond the seam were measured as well. Two of them contradict the received objection. 54% of the Rust clean build is third-party code. The project's own source compiles in 0.8 times the CPU `g++` needs. The 6.8× binary-size gap

is a linkage policy that reverses when the policy is held fixed. The compared component has no third-party crates and no `unsafe` sites (0 crates, 0 `unsafe` sites).

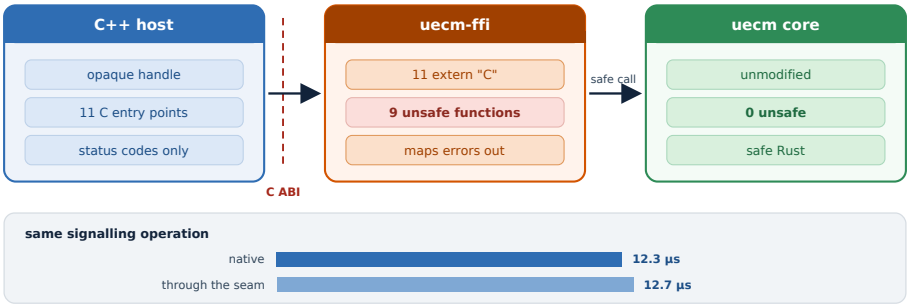


Figure 5.7: The C++-to-Rust FFI migration seam. The same signalling operation costs 12.3 µs directly and 12.7 µs through the seam, a paired difference bounded at 10.91% of one operation. A do-nothing crossing costs 2.54 ns and bounds only itself.

6 Discussion

6.1 Summary of findings

The answer to RQ1 divides at saturation. Below it the variants agree to 2 μs at the median, and the far tail resolves in Rust's favour. The end-to-end round-trip is only 0.250% core operation, so it cannot bound a language effect. The same operation timed apart is 1.6× faster in Rust. Past saturation they diverge in both directions. The Rust arms hold their offered rate to 448 kops/s against 392. At 784 kops/s C++ sustains 1.7× the throughput of Rust. RQ2 is final and does not depend on the host. All seven in-guarantee classes are compile errors in safe Rust. All seven boundary classes compile and misbehave in both languages. RQ3 is a memory penalty of 1.4–2.5 MiB that reproduces under both allocators and on all three machines, with CPU that inverts across the two load regimes. RQ4 is a small seam whose per-call tax is bounded at 10.91% of one operation. The real cost is API design and the learning curve.

6.2 Discussion on the research questions

What a deployment should do with RQ1 depends on where it expects to sit, not on a single ranking. A service sized to run below each arm's knee should prefer the arm that holds its offered rate further. One that a signalling storm will drive past each knee should prefer the arm that still completes more work afterwards. Those two preferences point at opposite languages. Only the second is tested. The first is descriptive, for the reason Section 5.3.2 gives. A comparison that stopped at a load the service still meets would have read the opposite with the same apparatus.

The closest prior work measures this quantity for 5G control-plane procedures. It finds the same shape and traces it to stackless `async`/`await` [32]. That attribution is accepted here. The shape appeared with no asynchronous runtime on either side and the concurrency model held fixed. A runtime cannot be the only place such an effect originates. The prediction registered in advance also named where to look first, bounds and overflow checking [51, 52]. That part of it was wrong. The median gap is consistent with evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [12, 35]. It is also weaker than it looks. The core operation is 0.250% of what the sweep measures.

On safety the difference is one of kind. The boundary is reported as a first-class result, not as a caveat. A compiler that refuses to build a program rules the defect out of each execution. A sanitiser rules it out only of the runs a sanitiser test happens to take. What is absolute there is the execution coverage, not the breadth of the catalogue. Figure 6.1 names the mechanism on each side of that line. Rust reasons about aliasing and not about acquisition order. A `Mutex` guard whose lifetime the borrow checker tracks perfectly says nothing about whether two threads take their locks in a compatible order. Static deadlock detection for Rust is an active research problem [53]. The lost update escapes ThreadSanitizer for the same reason it escapes the borrow checker. Where each access is correctly synchronised, there is no unsynchronised access to find. In a field study of 105 real concurrency bugs, about a third of the non-deadlock cases violated the order the programmers intended rather than their locking [54]. Adopting Rust retires one category of concurrency defect and leaves untouched the category that fails silently. A migration plan reading "memory-safe" as "concurrency-safe" would mis-budget its review effort. Memory-safety bugs in Rust involve `unsafe` in every published Common Vulnerabilities and Exposures (CVE) entry but one compiler bug [55], which is why keeping the core `unsafe`-free matters, and why unsafety spreading through dependencies is a review concern [56].

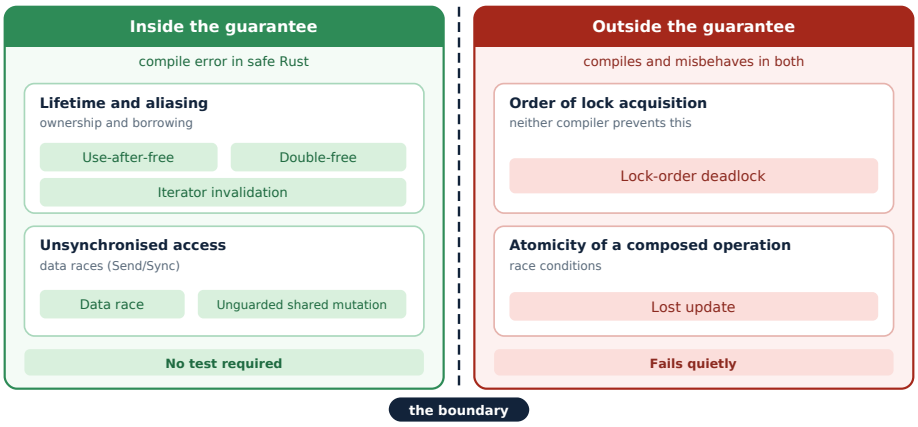


Figure 6.1: Where Rust's compile-time guarantee stops, with classes grouped by the mechanism that rejects them.

One case reaches past the injected programs. A specification gap let an operation be submitted outside the service's running window. It survived each test, each parity vector and all three sanitisers because nothing ever called that path. What turned undefined behaviour into a diagnosable failure on the Rust side was an ordinary bounds check, not a proof. The property worth budgeting for is "checked by default", not "ruled out at compile time". The root cause belongs to the method. A prohibition stated without a mechanism

behind it leaves each version to fail in its own idiom.

RQ3 splits by resource and the splits do not point the same way. The memory penalty reproduces under both allocators and on a second machine with a different compiler. Peak resident set is a run-level maximum, so the jitter that destroys a p99 does not touch it. Sized against the allocator's own effect, the two are of the same order on both bare-metal hosts. On the guest the allocator's was an order of magnitude larger. The language effect travels. The ratio does not. The quiet host also locates the cost outside the allocator, by the allocator's own accounting. One close study reports the other way sign, with Rust lowest and the gap widening as concurrency rises [29]. Those two findings agree once the arms are read as two concurrency runtimes. Their C++ runs on Boost.Fibers, which allocates a stack per fiber. Their Rust runs on Tokio, where a task is a small heap-allocated future. What they measured is the per-context cost of two concurrency runtimes. Two further studies agree with the direction found here [30, 57]. On CPU the sign depends on the load. A single number for "the CPU cost of Rust's safety model" would average two regimes that move in opposite directions.

RQ4 comes out cheap in the machine and expensive in the people. A C++ host can call an unchanged Rust core across a thin C-ABI seam whose cost on a real operation is bounded at 10.91% of one operation. The dominant cost is human [58, 59]. Confining `unsafe` to the seam bounds what has to be reviewed, not what can be corrupted. Host and component share one address space [60, 61]. The build and footprint axes point the same way as the seam. A vendor worried about either should audit its dependency budget before blaming the compiler. The one axis where C++ leads outright is the edit-compile loop for changes that stay inside a translation unit. The 6.8× binary-size figure compares two linkage policies, and the ordering reverses when the policy is held fixed.

6.3 Revisiting the hypotheses

Section 1.4 committed to three hypotheses before any measurement. Stating their disposition is what separates a test from a description.

H1 The median clause holds at the representative load. The far-tail clause does not, and overload, which the wording never named, is where the languages actually part. The reported campaign now resolves that tail, in Rust's favour, and a second bare-metal host agrees on the direction. H1 was right about where a below-saturation difference would show up and wrong that it was the only place. It was also not well posed. "Performance-equivalent" reads as one quantity and names at least three: the

delivered round-trip below saturation, the code the compilers emit, and sustained work under overload. Those three do not move together.

H2 For the seven injected classes of its kind it holds without exception. It fails in the looser reading its wording invites. It is accepted for memory safety and data-race freedom, rejected for concurrency correctness at large, and it is a claim about *safe* Rust only. The seam case shows the same class returning inside `unsafe`.

H3 Any per-operation tax is bounded at 10.91% of one operation, which is support from a bound on a cost the measurement could have missed, not from the failure to detect one. The accompanying claim, that the real cost is design and training, is supported structurally and not statistically.

6.4 Engineering guidelines for incremental adoption

The guidelines are the practical form of the findings, and each carries the limit of the finding behind it. Choose a first component for the narrowness of its interface, not for its size. The redesign of a rich interface is the schedule risk. The boundary itself is not. Keep the C ABI in its own crate and the core free of `unsafe`. That keeps unsafety out of the dependency chains it otherwise propagates through [56]. Leave the concurrency review budget where it is. Prefer a shared-nothing structure in either language. The strongest concurrency result here belongs to the design and to neither compiler. Plan the learning curve as a training cost and instrument it. Measure the component as well as the round-trip. Sweep past the point where the service stops coping. That is the guideline this study would most like to have followed from the start.

6.5 Where this study sits, and what it cannot support

The studies closest to this one are the five vendor-hosted theses of Section 2.3. Their verdicts contradict each other in each direction. The one reporting the other way sign on peak memory reconciles with this study once its arms are read as two concurrency runtimes rather than two languages. Three threats stay load-bearing, and the first is that one author wrote both versions. The same failure mode reversed a published verdict in this industry: comparable after a code review and then Rust worse by 85–91% after further optimisation [33]. The parity suite bounds behaviour and not performance. The

end-to-end instrument measures the design faithfully and the language barely at all. The overload divergence is replicated here with its mechanism narrowed and not closed, and no peer-reviewed study reports the same reversal.

6.6 Ethical and professional reflection

The study processes no personal, subscriber, or operator-confidential data. All traffic is synthetic and the subsystem is modelled on live code rather than copied from it, so the artefacts can be released openly. The first responsibility is not to over-claim. A vendor could read the RQ2 result as licence to describe a migrated component as “safe”, and the boundary cases are why that reading would be wrong. A deadlocked control plane is an outage. A silently lost counter update can misinform a mobility decision. Nothing demonstrated here prevents either. The second responsibility is how far the evidence reaches. Tail-latency numbers from any single machine are a narrower basis for a production commitment than they look. That is why the earlier guest campaign is reported rather than discarded.

7 Conclusion and Future Work

The thesis compared modern C++ and Rust. The object is a UE Context Management service modelled on a real Nokia gNB-CU-CP component. It looked at four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and cost of adding Rust a piece at a time. Both versions were built against one frozen specification and driven along the same code paths. That is what lets a language effect be told apart from a difference of design or of skill.

On safety the difference is one of kind and it has a clear boundary. Safe Rust rejects all seven injected memory-safety and data-race classes at compile time. Hazards that C++ can only hope to catch under a sanitiser test become programs that cannot be written. All seven boundary classes compile and misbehave in safe Rust as they do in C++. On two of them the C++ tooling reports what Rust's does not. Stated precisely, Rust eliminates data races and aliasing errors. It does not eliminate race conditions, lock-ordering errors, leaks, wrapping arithmetic or the blast radius of a failed worker, and it eliminates nothing inside an `unsafe` block. Where the boundary defects are caught, they are caught at test time. In two of the three cases that happens only after the defect is rewritten into a model checker's primitives.

On migration the cost is dominated by API design and the learning curve, not by any run-time tax. An unchanged Rust core can be driven from C++ across a thin seam. The cost on a real operation is bounded at 10.91% of one operation. All of the `unsafe` sits in the seam, and the core is verified free of it. Most of the Rust build's extra expense is compiling dependencies, not the compiler's own work. The binary-size gap is a linkage policy. The ordering depends on which matched policy is chosen. Where C++ leads is the edit-compile loop, and it leads only for edits that stay inside a translation unit.

On performance the evidence first divides by what is being measured. An end-to-end round-trip here is 0.250% language. The rest is an operating-system handoff. An end-to-end null bounds nothing about either compiler. The same operation measured apart is 1.6× faster in Rust. At the fixed load the paired test resolves 1.3 μs in Rust's favour, too small for a deployment to care about at the median. At p99.99 the slowest Rust variant sits 50 μs below the fastest C++ one. The Rust arms then hold their offered rate to 448 kops/s against 392. Once all four have collapsed, C++ sustains 1.7× the throughput of Rust, which spends 1.8× the CPU on each operation it completes. On memory the Rust build carries a few MiB more fixed footprint under both allocators, in the same direction

and scale on all three machines measured. The allocator choice costs about as much on both bare-metal hosts, and an order of magnitude more on the guest this study used first.

Taken together these support a narrow claim. It is for stateful, concurrent, partitionable control-plane services of this kind. At the loads where the service still meets its offered rate, the choice is which classes of defect a team wants to make unwriteable. The other half is whether it will pay the design and training cost of getting them. Past that point it is a speed decision as well. What the thesis adds is an instrument, not a verdict. That instrument is the frozen specification and the parity-tested pair. It is also a timing framework that reaches past the saturation knee. A split of the round-trip says how much of what it times the language can own. A defect catalogue maps where the guarantee stops.

Future work answers the gaps one at a time. The first job needs no new machine: vary the transparent-hugepage policy on one host with everything else held fixed. All three hosts agree that the Rust build carries the larger footprint. They disagree about how that compares against the cost of the allocator. The guest put it an order of magnitude higher, and neither bare-metal host does. The hugepage policy is the candidate this study can name but not test, because it differed between the hosts and only the system-allocator arm inherits it. Beyond that, replaying anonymised production traces would check the operation mix. The framework can also take further control-plane subsystems or the data-heavy layer-2 and physical layer (PHY) path. A subsystem with truly shared mutable state would exercise the two languages differently than a shared-nothing design does. Testing Rust async runtimes against the thread-per-shard model would separate the runtime from the language in the tail result [32]. Timing what in-process isolation of the two heaps costs would price what protecting an `unsafe`-free core is worth [60, 61]. A longer case study tracking defect counts and review effort while adding Rust a piece at a time is what would settle the human cost this study can only name.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, through the Cursor development environment, is the only generative AI system used in preparing this thesis, from June 2026 until submission. University of Helsinki policy forbids naming a language model as an author. It is not cited as a source either, which is the author's own choice.

What the assistance covered. The model drafted and revised the running prose, and proposed the order of the arguments. It wrote and refactored both versions of the UE Context Management service. It also wrote the load generator, the fault-injection catalogue, and the FFI prototype. The analysis behind the bootstrap intervals and the paired permutation tests is its work too, as are the figure, build and LaTeX sources. It suggested papers to read. The abstract is this programme's maturity test. University policy never permits a language model in one. The abstract was written by the author unaided.

What the model did not settle. The research problem, the four research questions, the three hypotheses, and the study's main control are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only planned gap between the two versions. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author, on the machines each result names. The quantities the thesis quotes reach the page as macros rebuilt from the raw logs. The readings are the author's too. A sweep dominated by cross-thread handoff cannot bound a language effect. An unresolved comparison is not a demonstration of equivalence. An operating point noticed after the data was in hand is described, not tested.

How the output was checked. All model output, prose and code, was read by the author before it entered the thesis. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body. An audit then graded how far each claim could be traced. The first grade is the page or clause itself. The second is a publisher-deposited record that carries the claim. The third is confirmed metadata behind a paywall. An entry whose supporting passage was never opened is listed, not absorbed. The audit found four passages that overstated or misplaced their source. One was a quotation absent from the work it was attributed to. All four were corrected. The audit is kept beside the artefacts. Code was checked against a conformance suite whose transcripts are diffed line by line. It was also checked against a fault catalogue that fixes each expected outcome before the defect is injected.

Responsibility. The author has reviewed the whole of this thesis and is responsible for all of it. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index

reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[26] A. Meneely, A. Green, T. Jaafari, M. Fluet, and B. Keller. ""Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C". In: *Proceedings of the 3rd IEEE/ACM International Workshop on Software Vulnerability Management (SVM)*. 2025, pp. 25–32. DOI: 10.1109/SVM66695.2025.00008. URL: https://doi.org/10.1109/SVM66695.2025.00008.

[27] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[28] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[29] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[30] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[31] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[32] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[33] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[34] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[35] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[36] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[37] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are

affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[38] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[39] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[40] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[41] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[42] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[43] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[44] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[45] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[46] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[47] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[48] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[49] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[50] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[51] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[52] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[53] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint

arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[54] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[55] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[56] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[57] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[58] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[59] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[60] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[61] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on*

Computer Systems (EuroSys). 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.